קטע קוד A

```
class MyStr1 {
    const char* str;
public:
    MyStr1(const char* s = ""): str(s) {}
    friend ostream& operator<<(ostream& out, const MyStr1& s) {
        return out << s.str;
    }
    const char* c_str() const { return str; }
};

class MyStr2 {
    size_t size;
    char* str;
    MyStr2(const char* s, size_t size)
    : size(size), str(new char[size+1]) { strcpy(str, s); }
public:
    MyStr2(const char* s = ""): MyStr2(s, strlen(s)) {}
    ~MyStr2() { delete[] str; }
    MyStr2(const MyStr2& s): MyStr2(s.str, s.size) {}
    MyStr2& operator=(const MyStr2& s) {
        if(this != &s) {
          delete[] str;
          size = s.size;
          str = new char[size+1];
          strcpy(str, s.str);
        }
        return *this;
    }
    operator const char*() const { return str; }
    size_t length() const { return size; }
    friend MyStr2 operator+(const MyStr2& a, const MyStr2& b) {
        MyStr2 retval(a, a.size + b.size);
        strcpy(retval.str + a.size, b);
        return retval;
    }
};
```

main A1

```
int main() {
  MyStr1 s1 = "hello";
  MyStr2 s2 = " world";
  auto s3 = s1.c_str() + s2;
  cout << s3 << endl;
}
```

main A2

```
int main() {
  MyStr1 s1 = "hi";
  MyStr2 s2 = "bye";
  auto s3 = s1;
  // MyStr2 s4 = s1.c_str();
  // cout << s4 << endl;
  // MyStr1 s5(s2);
  // cout << s5 << endl;
  // MyStr1 s6 = s3.c_str();
  // cout << s6 << endl;
}
```

main A3

```
int main() {
  MyStr1 s1(MyStr2("thing1"));
  // cout << s1 << endl;
  MyStr2 s2 = MyStr1("thing2").c_str();
  // cout << s2 << endl;
}
```

קטע קוד B

```
std::pair<size_t, std::map<size_t, size_t>>
fill(size_t big_bucket,
        const std::vector<size_t>& buckets,
        size_t index) {
  auto curr = buckets[index];
  if(big_bucket % curr == 0)
    return {0, { {curr, big_bucket / curr} }};
  if(index < buckets.size()-1 && big_bucket >= buckets.back()) {
     auto times = big_bucket / curr + 1;
     while(times > 0) {
       --times;
       auto rest = big_bucket - times * curr;
       auto result = fill(rest, buckets, index + 1);
       if(result.first == 0) {
         if(times > 0) result.second[curr] = times;
         return {0, result.second};
       }
     }
  }
  return {big_bucket, {}};
}

void printfill(size_t big_bucket,
               const std::vector<size_t>& buckets) {
  cout << "Trying to fill " << big_bucket << " with: ";
  for(auto small: buckets) {
    cout << small << ' ';
  }
  auto result = fill(big_bucket, buckets, 0);
  if(result.first == 0) {
    cout << "- can fill!!" << std::endl;
    for(const auto& fill_bucket : result.second) {
      cout << "bucket: " << fill_bucket.first
           << ", times: " << fill_bucket.second << endl;
    }
  }
  else cout << "- cannot fill :-(" << endl;
}

int main() {
  // try to fill big bucket of size 12 with: 5
  printFill(12, {5});
  // try to fill big bucket of size 10 with: 7, 5, 4
  printFill(10, {7, 5, 4});
  // try to fill big bucket of size 10 with: 7, 6, 5, 4
  printFill(10, {7, 6, 5, 4});
}
```

קטע קוד C

```
 1. class MyStr {
 2.   int size;
 3.   char* str = nullptr;
 4.   // private helpers
 5.   void stealFrom(MyStr&& s) {
 6.     cout << "in stealFrom, about to steal: "
 7.          << s << " into: " << *this << endl;
 8.     std::swap(size, s.size);
 9.     std::swap(str, s.str);
10.   }
11.   MyStr(size_t len): size(len), str(new char[size + 1]) {
12.     str[0] = '\0';
13.     cout << "in MyStr ctor, created: " << *this << endl;
14.   }
15. public:
16.   MyStr(const char* s = "") : MyStr(strlen(s)) { strcpy(str, s); }
17.   ~MyStr() {
18.     cout << "in ~MyStr about to delete: " << *this << endl;
19.     delete[]str;
20.   }
21.   MyStr(const MyStr& s) : MyStr(s.str) {
22.     cout << "in copy ctor after copying: "
23.          << s << " into: " << *this << endl;
24.   }
25.   MyStr(MyStr&& s) { stealFrom(std::move(s)); }
26.   MyStr& operator=(MyStr s) {
27.     stealFrom(std::move(s));
28.     return *this;
29.   }
30.   friend MyStr operator+(const MyStr& l, const MyStr& r) {
31.     MyStr s(l.size + r.size);
32.     strcpy(s.str, l.str);
33.     strcpy(s.str + l.size, r.str);
34.     return s;
35.   }
36.   friend ostream& operator<<(ostream& out, const MyStr& s) {
37.     return out << (s.str? s.str : "nullptr");
38.   }
39. };
40.
41. int main() {
42.   MyStr s1 = "Hello";
43.   cout << s1 + " world!" << endl;
44.   // s1 = "world " + s1;
45.   // cout << s1 << endl;
46.   // MyStr s2 = "world " + s1;
47.   // cout << s2 << endl;
48. }
```

בהצלחה !